BERT

Đọc hai chiều, đặt head và căn nhãn

Ngữ cảnh quyết định nghĩa của token

“không hay” khác “không tệ”.

Một token cần biết quan hệ với những token bên cạnh.

Từ bảng embedding cố định sang hidden state theo ngữ cảnh.

Câu hỏi: token nhìn được ai và đặt nhãn ở đâu?

Mục tiêu

- Hiểu cách BERT được huấn luyện (masked language modeling).
- Theo dõi shape từ token tới đầu ra cho cả câu và cho từng token.
- Huấn luyện một encoder nhỏ từ đầu.
- Gán nhãn từng token (NER) khi một từ bị tách thành nhiều subword.

01

Biểu diễn có ngữ cảnh

Cùng token, khác câu

“đường đi” và “đường ngọt”.

Embedding đầu vào: cùng một hàng bảng tra.

Hidden state sau encoder: khác nhau theo câu.

Position embedding bổ sung thứ tự.

BERT là một model và một cách huấn luyện cụ thể, không phải mọi encoder.

HỎI NHANH

Để phân biệt “đường” là lối đi với “đường” là vị ngọt, một bạn lấy `token_embedding.weight[id("đường")]` của checkpoint BERT làm vector của từ trong từng câu. Cách này cho ra gì?

A Hai vector khác nhau, vì BERT là model hai chiều

B Hai vector giống hệt, vì đó chỉ là một hàng của bảng tra

C Hai vector khác nhau, vì đã cộng position embedding

D Code báo lỗi, vì embedding chỉ đọc được bên trong `forward`

Một attention head

$$\frac{}{\sqrt{}}$$

Softmax theo key; liên kết bị cấm có mask .

Một head: shape `(B, L, d_k)`, shape `(B, L, d_v)`.

Ma trận trọng số shape `(B, L, L)`.

HỎI NHANH

Câu có 3 token thật, được pad thêm 2 PAD thành độ dài 5. Ở hàng attention của token thật thứ hai, encoder BERT và decoder causal lần lượt cho phép bao nhiêu key?

A Encoder 3, causal 2

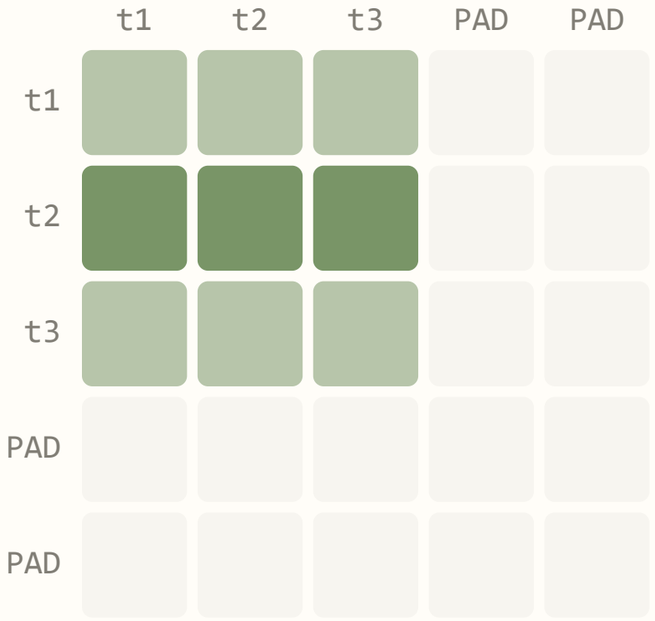
B Encoder 5, causal 2

C Encoder 3, causal 3

D Encoder 2, causal 3

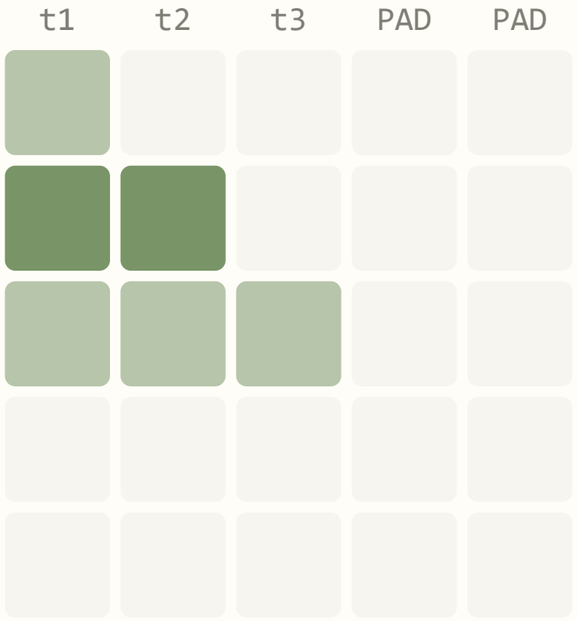
Hai chiều và causal khác nhau

encoder BERT: chặn PAD



hàng t2: 3 key

decoder causal: chặn PAD và tương lai



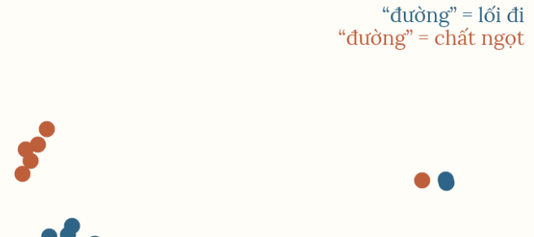
hàng t2: 2 key

Encoder chỉ chặn cột PAD; causal mask còn chặn key tương lai.

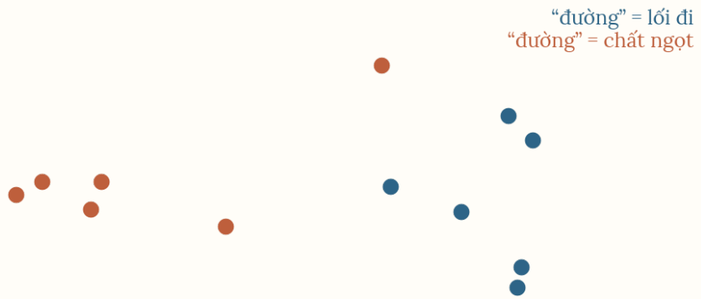
Một chữ, hai nghĩa, qua từng lớp



lớp 0, embedding tra bảng: cosine trung bình cùng nghĩa 0.76, khác nghĩa 0.77



sau lớp 3: cosine trung bình cùng nghĩa 0.69, khác nghĩa 0.66



sau lớp 7: cosine trung bình cùng nghĩa 0.61, khác nghĩa 0.36



sau lớp 12: cosine trung bình cùng nghĩa 0.46, khác nghĩa 0.13

Vector của chữ “đường” trong 12 câu: càng lên lớp cao, hai nghĩa càng tách xa.

Masked language modeling

“An [MASK] sách”: đoán token gốc “đọc”.

Loss chỉ ở 15% vị trí đã chọn.

Trong đó: 80% `[MASK]`, 10% ngẫu nhiên, 10% giữ nguyên.

`[MASK]` không có khi fine-tune, nên model phải đọc tốt mọi token.

Pretraining: corpus không nhãn. Fine-tuning: nhãn câu hoặc token.

RoBERTa: bỏ next sentence prediction, chọn lại vị trí che mỗi lượt.

HỎI NHANH

Một vị trí được chọn cho MLM chứa token gốc “đọc”, và ở lượt này đầu vào tại đó được thay bằng `[MASK]`. Nhãn của vị trí ấy là gì?

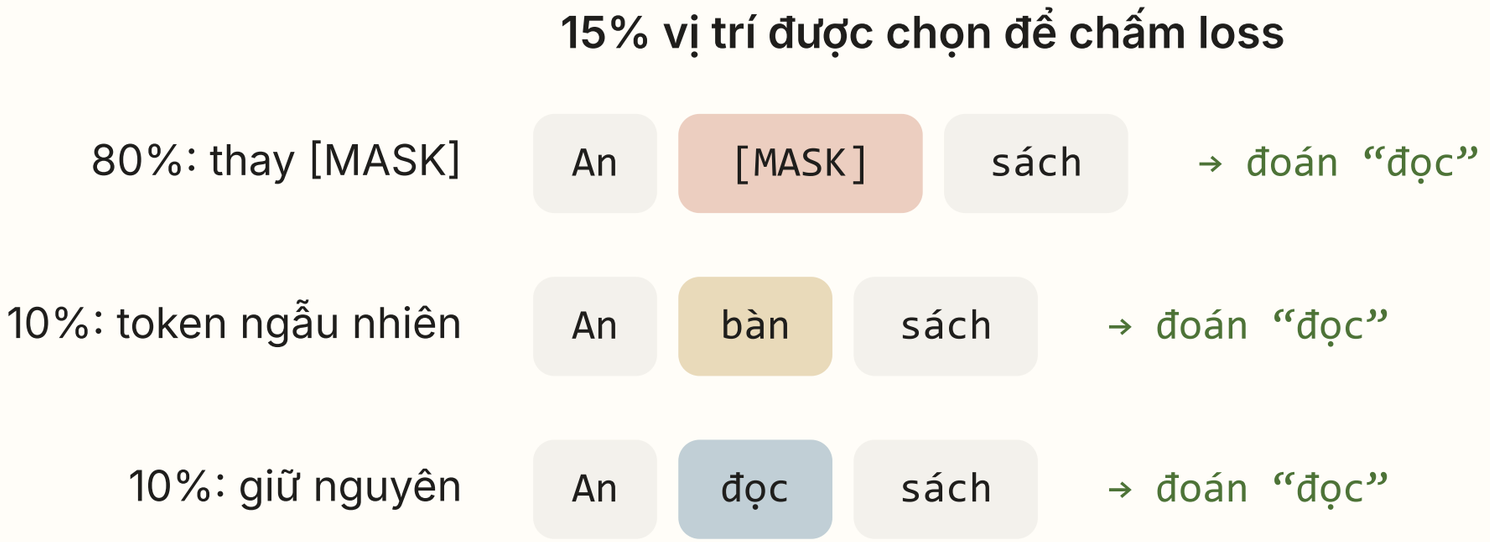
A ID của `[MASK]`, vì đó là token model nhìn thấy

B `-100`, vì vị trí đã bị che thì bỏ khỏi loss

C Nhãn cảm xúc của cả câu chứa vị trí ấy

D ID của “đọc”, token gốc trước khi bị che

Che 15% vị trí



nhãn luôn là token gốc; vị trí không được chọn mang -100

BERT chọn 15% vị trí: 80% thay MASK, 10% thay token ngẫu nhiên, 10% giữ nguyên.

HỎI NHANH

Một câu có 40 token thật và 24 PAD. MLM chọn 6 vị trí, và tổng của token gốc trên 6 vị trí ấy là 9.0. của câu bằng bao nhiêu?

A 0.1406

B 0.225

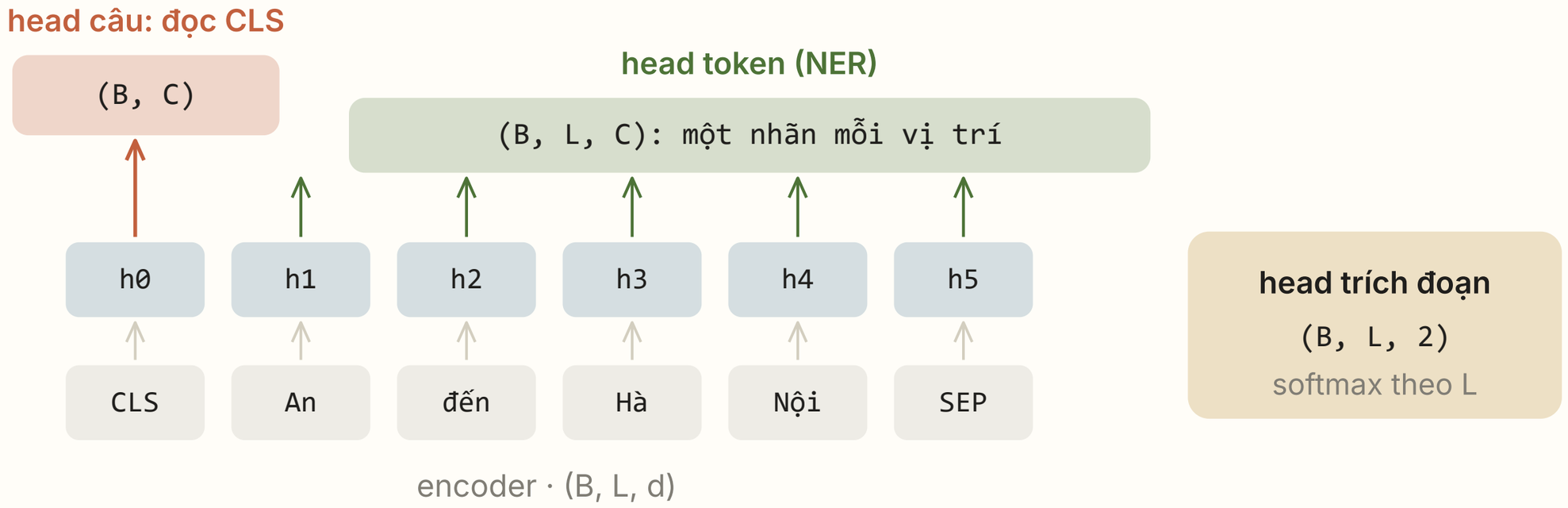
C 1.5

D 9.0

02

Một encoder, hai head

Head câu và head token khác số đầu ra



Ba cách đặt head: đọc CLS cho cả câu, mỗi token một nhãn, hoặc điểm đầu và cuối của đoạn.

Shape cần theo dõi

Tensor	Shape
IDs	`(B, L)`
hidden	`(B, L, d)`
head câu	`(B, C)`
head token	`(B, L, C)`

Logits đưa thẳng vào cross-entropy.

Head nhiệm vụ mới thường là ngẫu nhiên.

Trích đoạn (QA): `(B, L, 2)`, softmax theo .

HỎI NHANH

Head trích đoạn trả về logits `(1, 384, 2)`. Argmax của logits “bắt đầu” rơi vào vị trí 57, còn argmax của logits “kết thúc” rơi vào vị trí 41. Đọc đáp án thế nào?

A Đảo hai đầu, lấy đoạn từ vị trí 41 đến 57

B Chọn cặp có tổng hai điểm lớn nhất

C Trả về rỗng, vì hai argmax không tạo được đoạn hợp lệ

D Chỉ lấy token 57, vì điểm bắt đầu chắc hơn

03

Encoder nhỏ trên CPU

Một encoder nhỏ học từ đầu

1 lớp encoder, hidden size 16, 2 heads.

10 câu train tự soạn, 2 câu validation.

Giống một lớp BERT: embedding token cộng vị trí, attention hai chiều, head đọc CLS.

Bỏ giai đoạn MLM: học thẳng nhãn từ khởi tạo ngẫu nhiên.

Nhãn 1: “hay”, “không tệ”. Nhãn 0: “tệ”, “không hay”.

Padding mask đúng quy ước

```
hidden = self.encoder(
    inputs,
    src_key_padding_mask=ids.eq(0))
```

PyTorch key padding mask: **True bỏ**.

HF attention mask: **1 giữ**.

Không truyền nguyên mask giữa hai API khác quy ước.

HỎI NHANH

Batch `["bài không hay", "giảng hay"]` có ID `[[1, 3, 6, 5], [1, 4, 5, 0]]`. Một bạn truyền mask kiểu HF `ids.ne(0)`, tức True ở token thật, thẳng vào `src_key_padding_mask`. Chuyện gì xảy ra?

A Không có gì thay đổi, vì True ở hai quy ước cùng nghĩa là giữ

B PyTorch tự đảo mask khi nhận tensor kiểu bool

C Câu hai chỉ còn đọc PAD, còn câu một mất hết key

D Chỉ CLS bị bỏ, các token khác vẫn đọc đúng

Kết quả thật còn chưa tốt

KẾT QUẢ

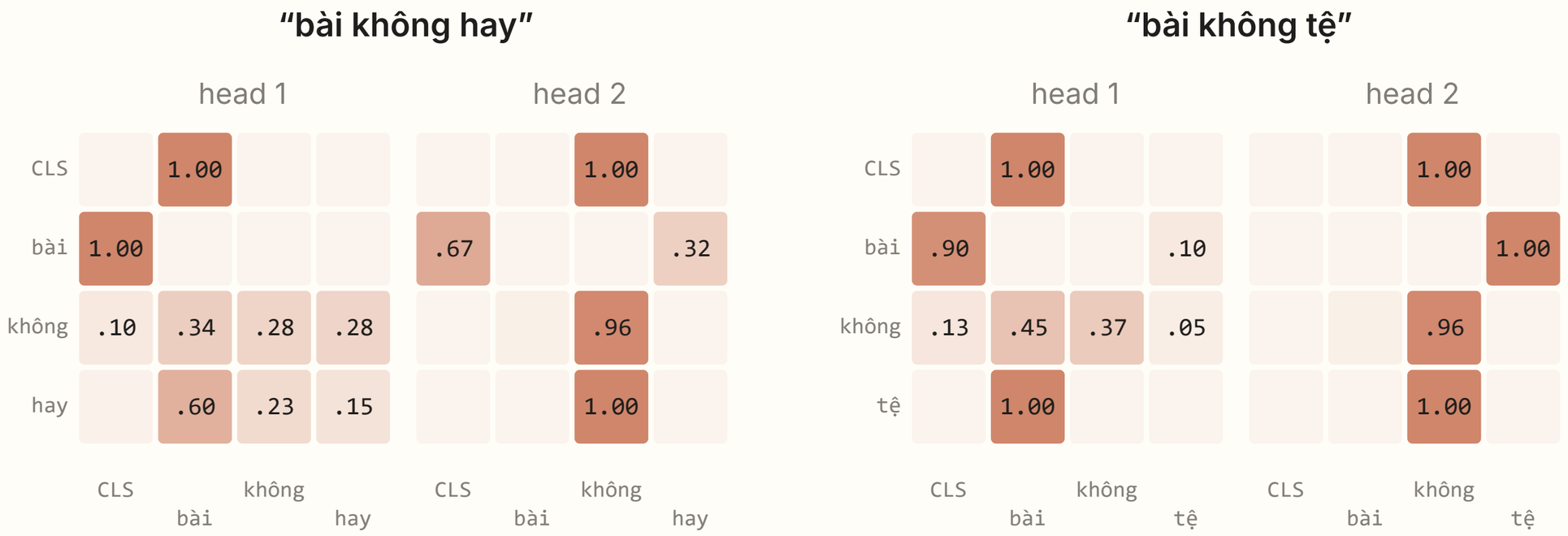
```
loss đầu/cuối: 1.248 0.2777
train accuracy: 0.8
giảng tệ thật: 0 đoán: 1
giảng không hay thật: 0 đoán: 1
```

Loss giảm chưa có nghĩa đã đọc được phủ định.

Train còn sai 2/10, validation sai 2/2.

: bốn câu đang ở xác suất 0.5?

CLS nhìn gì



hàng CLS của hai câu gần như trùng nhau: head đọc CLS không thấy khác biệt hay/tệ

Hàng CLS gần như giống hệt ở hai câu khác nhãn.

“không” đảo nghĩa: dạng XOR

KẾT QUẢ

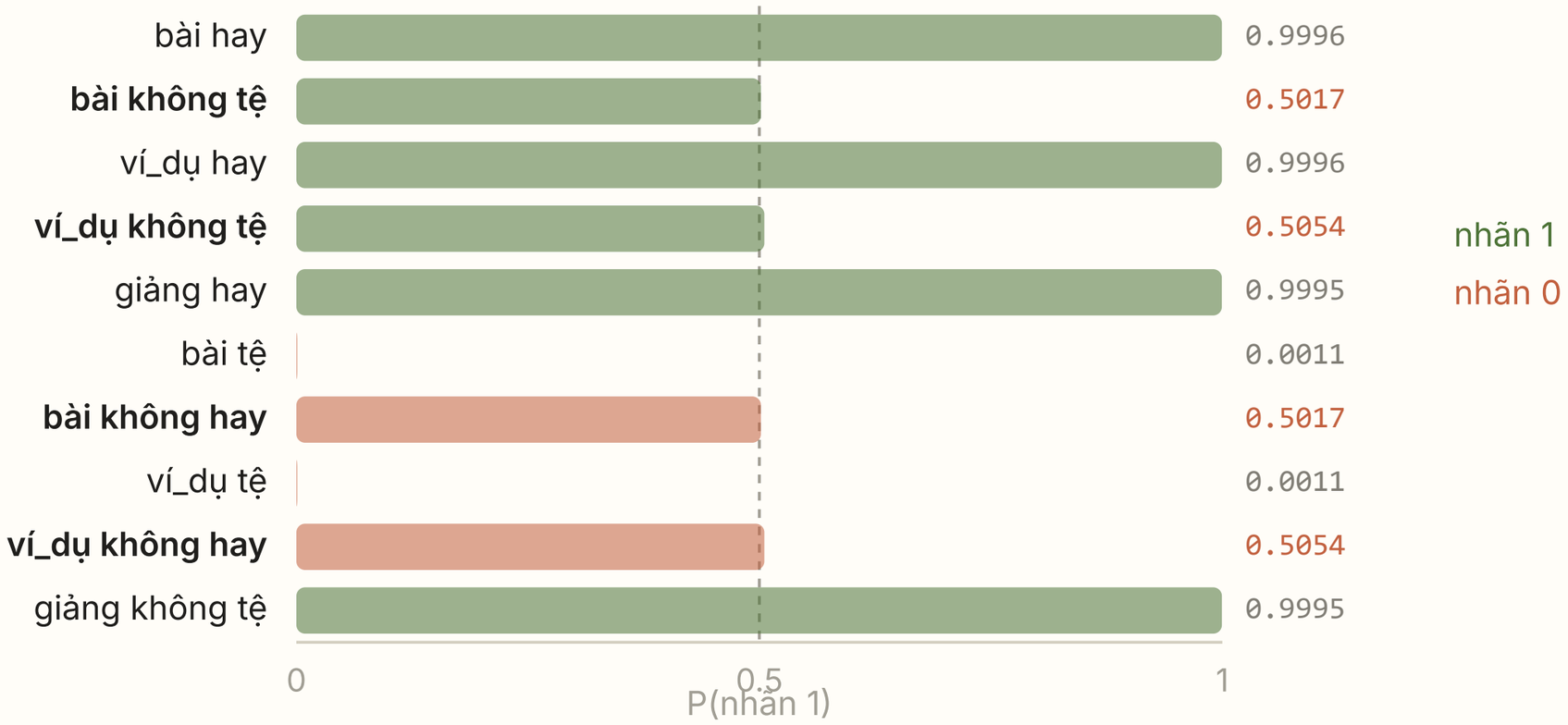
```
bài không tệ: nhãn 1, P(1) = 0.5017
ví_dụ không tệ: nhãn 1, P(1) = 0.5054
bài không hay: nhãn 0, P(1) = 0.5017
ví_dụ không hay: nhãn 0, P(1) = 0.5054
giảng không tệ: nhãn 1, P(1) = 0.9995
```

“không tệ” và “không hay” cùng xác suất.

Nhãn phụ thuộc **cặp** từ.

“giảng” chỉ đi với nhãn 1 trong train: lối tắt.

Xác suất của mười câu train



Sau 100 bước, bốn câu “không hay”, “không tệ” vẫn đứng ở 0.5.

Năm seed khác

KẾT QUẢ

```
seed 1: loss cuối 0.2781, train 0.8, validation đúng 0/2
seed 2: loss cuối 0.0000, train 1.0, validation đúng 2/2
seed 3: loss cuối 0.0001, train 1.0, validation đúng 0/2
seed 4: loss cuối 0.0000, train 1.0, validation đúng 2/2
seed 5: loss cuối 0.0001, train 1.0, validation đúng 1/2
```

Seed 2–5 cùng train 1.0, validation từ 0/2 đến 2/2: train không nói gì về “giảng tệ”.

Hai câu validation đo lối tắt, không đo phủ định.

Sáu seed, cùng cách train

seed	loss cuối	train	validation
63	0.2777	0.8	0/2
1	0.2781	0.8	0/2
2	0.0000	1.0	2/2
3	0.0001	1.0	0/2
4	0.0000	1.0	2/2
5	0.0001	1.0	1/2

kẹt ở 0.4 ln 2 hoặc học thuộc train; validation chỉ là may rủi trên hai câu

Cùng cách train, sáu seed cho kết quả rất khác nhau.

HỎI NHANH

Seed 2 đến 5 đều đạt 1.0 trên train, nhưng trên 2 câu validation lần lượt được 2/2, 0/2, 2/2 và 1/2. Nên làm gì tiếp?

A Nộp seed 2, vì đó là kết quả validation cao nhất

B Kết luận encoder hai chiều không học được phủ định

C Đổi learning rate rồi chọn lại seed trên hai câu này

D Thêm câu có “giảng” ở cả hai nhãn rồi mới so

Một bất biến để bắt lỗi

Thêm 3 PAD vào cuối câu, giữ đúng mask.

KẾT QUẢ

```
logits không đổi: True
```

PAD không làm key cho token thật.

Pooling và loss vẫn phải bỏ PAD riêng.

`eval()` và `no_grad()` khi kiểm.

HỎI NHANH

Attention mask đã đúng. Vì sao nhãn token ở vị trí PAD vẫn phải là `-100`?

A Vì mask chỉ chặn PAD làm key, còn loss vẫn tính ở mọi vị trí có nhãn

B Vì mask attention chỉ có tác dụng khi model chạy trên GPU

C Vì `-100` là token ID riêng mà tokenizer dành cho PAD

D Không cần, vì hidden state tại PAD bằng 0 nên loss bằng 0

04

Nhãn qua subword

NER giữ nhãn ở đơn vị từ

word_id	None	0	1	2	2	3	None
	[CLS]	An	đến	Hà	##Nội	sớm	[SEP]
nhãn	-100	B-PER	O	B-LOC	-100	O	-100

4 vị trí góp loss; ##Nội cùng word_id 2 với Hà nên nhận -100

Chỉ subword đầu của mỗi từ mang nhãn; CLS, SEP và subword tiếp nối nhận −100.

Hà_Nội bị cắt làm hai

Từ: An, đến, Hà_Nội, sớm.

Nhãn: B-PER, O, B-LOC, O.

Token: CLS, An, đến, Hà, ##Nội, sớm, SEP.

Chính sách: chỉ subword đầu có loss.

`-100`: bỏ khỏi loss, không phải token ID.

HỎI NHANH

Ta đổi chính sách: chép nhãn của từ cho mọi subword, nên `Hà` và `##Nội` đều mang `B-LOC`. Đọc dự đoán ở mức subword theo quy tắc BIO thì “Hà Nội” thành gì?

A Một thực thể LOC, vì hai subword cùng nhãn

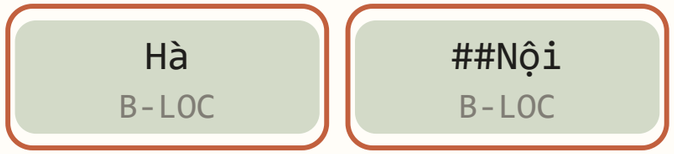
B Hai thực thể LOC là “Hà” và “Nội”

C Không có thực thể nào, vì thiếu nhãn `I-LOC`

D Một thực thể LOC chỉ gồm subword “Hà”

B mở thực thể mới

chép B-LOC cho cả hai



2 thực thể

đổi subword sau thành I-LOC



1 thực thể

Hai B-LOC liền nhau là hai thực thể; subword tiếp nối phải mang I-LOC.

Kết quả alignment

Nhãn token:

```
[-100, 1, 0, 2, -100, 0, -100]
```

KẾT QUẢ

```
vị trí tính loss: 4
loss: 0.2395
```

Logits **giả định**: lớp đúng 2, hai lớp kia 0.

.

HỎI NHANH

Quên đặt `-100`, nên ba vị trí `[CLS]`, `##Nội` và `[SEP]` nhận nhãn `0`, tức ID 0, còn logits của chúng vẫn là . Loss trung bình mới bằng bao nhiêu?

A 0.2395

B 0.6077

C 1.0635

D 1.0986

Sang checkpoint pretrained

Script: DistilBERT multilingual, head câu hoặc token mới.

Căn nhãn bằng `word_ids()`, subword đầu.

Mẫu vượt `--max-length` (tối đa 512): từ chối, không cắt.

AdamW , 2 epoch; `--head-only` như bài 4.3.

Chấm macro-F1 hoặc entity-F1 mỗi epoch, giữ epoch tốt nhất.

`run-receipt.json`: lịch sử, điểm tốt nhất, nhãn.

HỎI NHANH

Một hồ sơ NER dài 700 subword, và script dừng lại với thông báo mẫu vượt `max_length`. Xử lý thế nào để giữ được nhãn của cả hồ sơ?

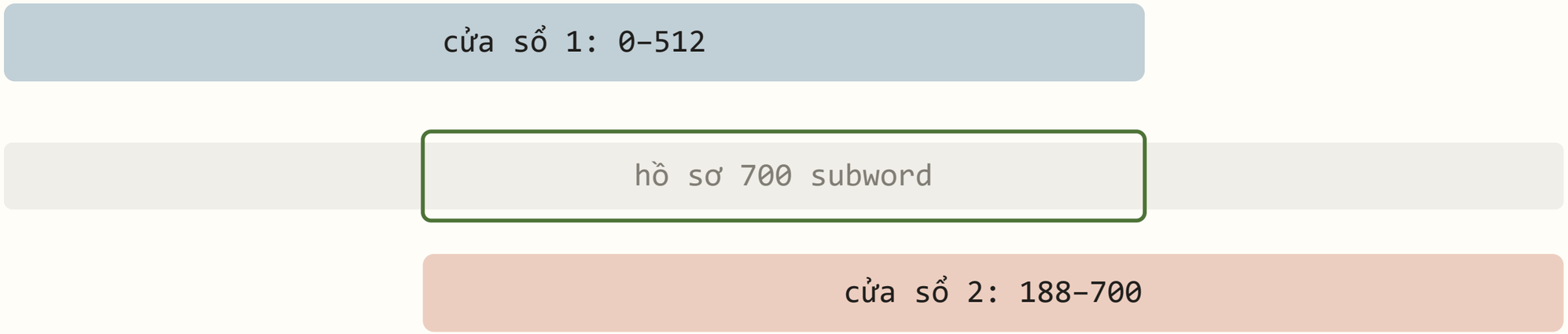
A Đặt `--max-length 1024` để chứa trọn cả hồ sơ

B Bật truncation để cắt hồ sơ còn 512 subword

C Bỏ hồ sơ này khỏi dữ liệu để script chạy tiếp

D Chia thành các cửa sổ tối đa 512, chồng lấp nhau, rồi ghép lại theo từ

Cửa sổ chồng lấp



vùng chồng lấp: giữ dự đoán của cửa sổ mà từ nằm gần giữa hơn

Văn bản dài chia thành các cửa sổ chồng lấp; mỗi từ lấy dự đoán từ cửa sổ mà nó gần giữa hơn.

Khi đi thi

- Đơn vị chấm: câu, từ hay đoạn trích.
- In tokenization và nhãn `-100` trước khi train.
- Learning rate nhỏ; chạy 2–3 seed.

05

Luyện đề

LUYỆN ĐỀ 1

Code MLM thay **toàn bộ** vị trí được chọn bằng `MASK_TOKEN`. BERT gốc làm gì ở những vị trí ấy?

A 80% thành `[MASK]`, 10% token ngẫu nhiên, 10% giữ nguyên

B 100% thành `[MASK]`, 80/10/10 chỉ là ablation

C Dùng một embedding `[MASK]` riêng cho gradient

D Chỉ che danh từ và động từ

ĐỀ THI NOAI 2026 Singapore · Chung kết · Câu 20

LUYỆN ĐỀ 2

Một đội có hai checkpoint cùng cỡ: một encoder hai chiều kiểu BERT và một decoder tự hồi quy kiểu GPT. Đề có hai phần: gán nhãn thực thể cho từng từ trong hồ sơ, và viết tiếp đoạn kết luận cho hồ sơ. Nên dùng model nào cho phần nào?

A Dùng BERT cho cả hai phần

B Dùng GPT để gán nhãn, BERT để viết tiếp

C Dùng BERT để gán nhãn, GPT để viết tiếp

D Dùng GPT cho cả hai phần

PHỎNG THEO NOAI 2026 Singapore · Vòng sơ loại · Câu 224

LUYỆN ĐỀ 3

Một đội thay encoder BERT bằng một decoder causal cùng cỡ, nhưng vẫn giữ `logits = head(hidden[:, 0])`, với `[CLS]` vẫn đứng đầu mọi câu. Sau khi fine-tune, gần như chắc chắn chuyện gì sẽ xảy ra?

A Vẫn tốt như cũ, vì `[CLS]` luôn tóm tắt cả câu

B Tốt hơn, vì decoder được pretrain trên nhiều chữ hơn

C Mọi câu ra cùng logits, model chỉ đoán được lớp đông nhất

D Code báo lỗi shape, vì decoder không có token `[CLS]`

PHỎNG THEO NOAI 2026 Singapore · Vòng sơ loại · Câu 229

Chốt ý

Hai chiều hay causal, padding mask và `-100`: mỗi thứ chặn một chỗ khác nhau.

Head câu, head token, head trích đoạn: đọc đúng shape; dữ liệu nhỏ thì chạy vài seed.

Sau giờ học

Làm **10 câu kiểm tra cuối bài** trên trang bài học.

Chạy notebook; đổi chỗ “giảng tệ” (sang train) với “bài tệ” (sang validation), rồi so lại năm seed.